

LearnLoop Project Report

AI Powered Quiz Generator and Learning Gap Analyzer

A browser-based study workflow that uses Groq to generate source-linked question drafts, keeps review in the learner-facing interface, and tracks quiz attempts and revision topics.

PROJECT DETAILS	SUBMISSION INFORMATION
Project domain	Generative AI for education
Student name	Aadi Ratn
Roll number	23051560
Batch	GEN AI 2026
Submitted to	Sachin Lohar
Submission date	30 September 2026

1. Problem Statement

The primary user is a learner studying from class notes or a selectable-text PDF. Learners need practice tied to their supplied material and a clear way to identify topics to revisit. Manually writing questions takes time, while unchecked model answers can be unsupported or ambiguous.

A workflow run meets the software success criterion when source material creates a job, Groq can return drafts that pass the implemented structure and evidence checks, a learner approves questions before practice, submitted answers are scored against the reviewed key, and progress can be exported. Verification recorded 20 passing core tests, 12 passing scripted UI checks, and one hosted Groq request that imported six drafts. These measures assess software behavior, not learning gains.

Table 1.1: User need and software success evidence

User	Need	Project success evidence
Students studying from notes or text-based PDFs	Prepare focused practice and see which named topics need another look	Local audit: 20 core tests and 12 scripted UI checks passed; a hosted Groq request generated six drafts.

2. Objectives and Scope

- Convert pasted notes or selectable text from a PDF into topic-labelled source passages.
- Create a question-generation job with a selected question count and recall or application style.
- Validate model output for matching job identity, JSON fields, four distinct choices, answer index, source quote and answer phrase.
- Require explicit human approval before each question can be used in a quiz.
- Score completed attempts deterministically and preserve the question and answer snapshot.
- Identify topics below the 70 percent threshold and offer practice using reviewed questions from those topics.

Table 2.1: Project scope

In scope	Out of scope
Hosted Streamlit learner interface with direct Groq question generation; pasted notes and text PDFs; manual review; local scoring, attempt history and weak-topic practice.	Scanned-PDF OCR; semantic proof of correctness; timed or proctored exams; learner accounts and cross-device sync; durable hosted storage; unseen revision-question generation; validated learning-gain or diagnostic claims; institution-scale deployment. See Section 11.2 for the limits of each.

3. Solution and Features

LearnLoop is designed as a hosted Streamlit learning app that students use directly in a browser. The app sends generation requests to Groq, receives question drafts, validates them locally, and presents them for source-based human review. SQLite stores jobs and attempts; scoring and topic summaries run locally in Python.

- Prepare: paste notes under topic headings or upload a text-based PDF; create and save a question job.
- Generate: the primary product path calls Groq directly from the Streamlit app. Learners use the hosted web interface to prepare notes, generate a pack and review questions. The Colab notebook and worker are development artifacts, not the intended end-user path.
- Review: inspect the source, edit question fields and explicitly confirm the answer and explanation.
- Practice: answer every item; keys and explanations are hidden until submission.
- Progress: review scores and topic breakdowns; Weak topics reuses reviewed questions below the threshold.

The saved demo bundle loads previously generated and reviewed questions without a new model call. It supports demonstration of the review, practice and progress flow.

4. Tech Stack

Table 4.1: Technology stack

Technology	Purpose	Reason in this project
Python	Application logic, parsing, validation and scoring	Keeps checks explicit and makes scoring deterministic.
Streamlit 1.45.1	Hosted web interface for direct learner interaction	Learners can prepare notes, generate questions and review them in one browser-based workflow.
SQLite	Session-scoped temporary storage in hosted use; configurable database file for local development.	Stores jobs, packs and attempt snapshots without a separate server.
PyPDF 6.15.0	PDF text extraction	Reads selectable text and retains page numbers for evidence.
Groq API	Live model inference from the Streamlit app	The server-side app sends generation requests; the key is configured as a Streamlit secret.
Qwen qwen/qwen3.8-27b	Question generation through Groq	Model selected by the current app code. Earlier Colab experiments used Qwen2.5-3B-Instruct.
Google Colab notebook	Development and recorded experiments	Retained in the archive; not the intended generation path for learners using the hosted app.

5. Knowledge Base and Data

Learners supply the source material. Pasted notes may use ## Topic headings. The parser divides content into passages of about 250 to 280 words and caps jobs at 30 passages. PDF ingestion accepts files up to 15 MB and 40 pages; pages with at least 80 characters of extracted text are retained with their page number. Scanned PDFs need OCR, which LearnLoop does not provide.

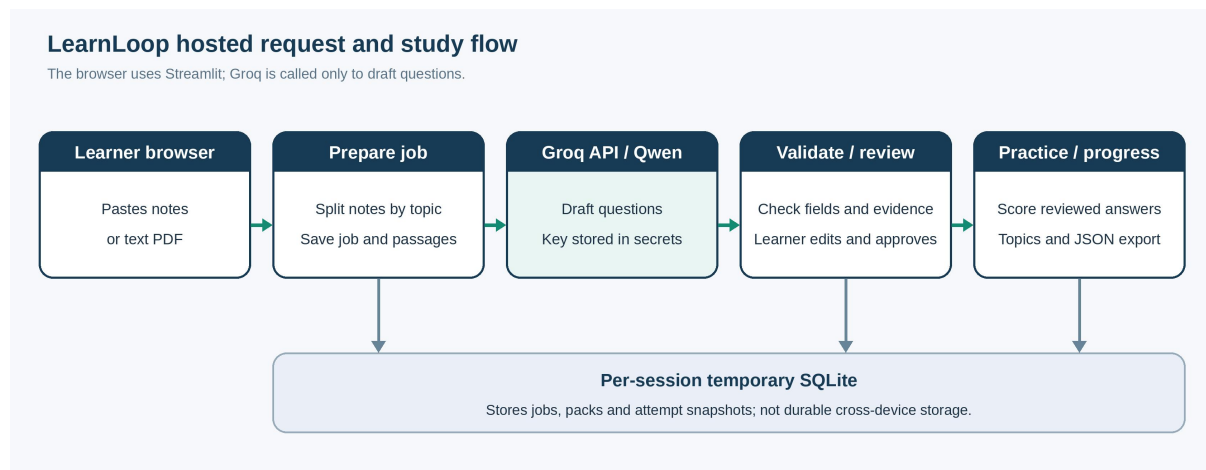
Table 5.1: Stored records and relationships

Record	Important fields and relationships
Source passage	Source ID, topic, document title, optional PDF page and passage text. Each question references a source ID.
Generation job	Job ID, title, difficulty, question items, embedded sources and SHA-256 checksum. The result must match job ID and checksum.
Question pack	Validated question drafts, source links, rejection reasons, model label and reviewed status. Each question is approved after edits.
Attempt snapshot	Pack/version, selected questions and source records, submitted answers, score and topic totals. Finished attempts are not rewritten.

SQLite uses a single records table keyed by kind and ID, storing each record as JSON. Study notes are included in saved jobs and packs, so exports and database backups should be treated as study material.

6. System Architecture

Figure 6.1: LearnLoop hosted request and study flow



The browser sends notes or a text-based PDF to the hosted Streamlit app. The app creates a job, calls Groq with Qwen, checks the returned structure and evidence, and presents drafts for human review. SQLite stores jobs, packs and attempt snapshots in a temporary per-session database. After approval, practice scoring and weak-topic calculations run locally without another model call.

Figure 6.1 separates the external model call from the learner workflow: Groq is used for drafting, while validation, approval, quiz scoring, progress summaries and exports run in the Streamlit application.

7. Implementation Highlights

7.1 Target fact prompt

The prompt asks for one question about a specified fact, disallows outside facts, requests JSON and requires a verbatim evidence quote. The worker selects source sentences and asks for a term-based answer. This constrains generation but does not prove that a question has one pedagogically sound answer.

```
target=facts[turn%len(facts)] if facts else source['text']
prompt='TARGET FACT: '+target+'\nWrite one '+job['difficulty'].lower()+ ' question asking
WHICH TERM fits this description. Use the exact subject phrase of the sentence as the correct
answer. Provide EXACTLY FOUR options, each a short term (not a sentence). Copy the TARGET
FACT as the quote. Explain the answer. For application style, use a short scenario.'
messages=[{'role':'system','content':job['system']},{ 'role':'user','content':json.dumps(prompt,ensure_ascii=False)}]
chat_completion = client.chat.completions.create(messages=messages, model="qwen/qwen3.8-27b",
temperature=0.3)
```

7.2 Validation boundary

```
if norm(q['quote']) not in norm(source['text']):
    raise ValueError('Evidence quote is not present in the source.')
if norm(options[correct]) not in norm(q['quote']):
    raise ValueError('Correct answer phrase is absent from the evidence quote.')
```

The importer also checks four nonempty, distinct options, a valid answer index, required fields, duplicate stems, and matching job ID/checksum. Invalid records are rejected with a reason; valid drafts still require human approval.

7.3 Deterministic attempt scoring

The grader compares each submitted option index with the reviewed correct index. It calculates total and per-topic results without a model call. SQLite preserves the attempt snapshot so later pack edits do not alter earlier scores. Weak topics fall below 70 percent on the latest completed attempt.

```
rows.append({'id':q['id'],'topic':q['topic'],'stem':q['stem'],'selected':q['options'][answer],
'expected':q['options'][q['correct']], 'correct':answer==q['correct'],'explanation':q['explanation'],
'quote':q['quote'],'source_id':q['source_id']})
topics=defaultdict(lambda: {'correct':0, 'total':0})
for row in rows:
    topics[row['topic']]['total']+=1;topics[row['topic']]['correct']+=int(row['correct'])
score=sum(r['correct'] for r in rows)
def weak_topics(result,threshold=70):
    return [t for t,s in result['topics'].items() if 100*s['correct']/s['total']<threshold]
```

8. Testing and Results

The local audit passed 20 core tests and 12 scripted UI checks for the Groq-enabled implementation, including the request adapter, missing-secret behavior, visitor session isolation, review, practice, progress and exports. A hosted Groq request to qwen/qwen3.8-27b successfully generated six draft questions for review. This confirms a live request worked at that time, not continuous service availability or educational correctness. Hosting checks found a Progress chart dependency error and low-contrast text; both were fixed and published. The core tests passed again after the chart fix.

Table 8.1: Objective-to-evidence check

Section 2 objective	Recorded result
Convert notes or selectable PDF text into topic-labelled passages.	Topic extraction and PDF page-evidence tests passed; OCR for scanned PDFs is not implemented.
Create jobs with a selected count and question style.	Included in the 12 scripted UI checks; the hosted smoke check generated six drafts for one job.
Validate identity, JSON fields, choices and source evidence.	Core tests passed for malformed output, duplicate choices, missing answer phrases and mismatched job/checksum.
Require approval before practice.	Unreviewed-question gate and review workflow passed the core and scripted UI checks.
Score deterministically and preserve the answer snapshot.	Scoring, restart persistence and idempotence tests passed.
Identify weak topics and offer revision practice.	Weak-topic scoring and revision tests passed; revision reuses prior questions and does not measure unseen-item mastery.

Table 8.2: Software test outcomes

Test group	Recorded result	Coverage or observed behavior
Core unit tests	20 passed	Malformed output, evidence and review gates, scoring, persistence, PDF pages, and a mocked Groq/Qwen request.
Scripted UI workflow	12 passed	Prepare, missing-secret warning, visitor isolation, review, quiz, progress, revision and saved-demo flows.
Saved model-output demonstration	Six saved drafts; six approved after review	Recorded earlier Colab output; not a live Groq request. Four questions were manually edited. Deliberate errors on two database items produced 4/6 (66.7%) in the classroom demonstration.
Revision demonstration	2 / 2 on reused questions	Targeted revision uses two previously reviewed questions; this demonstrates workflow behavior, not transfer to unseen material.

Table 8.3: Failure and workflow test cases

#	Test case	Expected behavior	Recorded result
1	Malformed model JSON	Importer rejects the malformed record and records the reason.	PASS in core test log
2	Answer phrase is absent from the source quote	Question is rejected by the evidence validator.	PASS in core test log
3	Result job ID or checksum does not match the saved job	Import is blocked before the result enters the pack.	PASS in core test log
4	Question has not been reviewed	Unreviewed question cannot be used for scoring.	PASS in core test log
5	Quiz is submitted with unanswered items	Submission is rejected; the attempt is not marked finished.	PASS in core test log
6	Restart app and resubmit a finished attempt	Saved history remains available; duplicate submission does not rewrite the result.	PASS in core test log
7	Six-question saved demo with two deliberate database errors	Score is 4/6 (66.7%) and Database fundamentals is identified for revision.	Observed in recorded UI run
8	Review, answer-key hiding, progress and weak-topic revision flow	Review approval is required; answer details stay hidden until submit; revision questions come from the flagged topic.	Included in 12 scripted UI checks
9	Groq secret missing	Show a clear warning and disable live generation without crashing the app.	PASS in scripted UI check
10	Two independent browser sessions	Use separate SQLite databases; one session cannot read the other session's jobs.	PASS in scripted session-isolation check
11	Groq request adapter	Construct a request for qwen/qwen3.8-27b and return one record per job item.	PASS with mocked Groq SDK; no live API call

9. Evaluation and Metrics

Results come from the local audit of the Groq-enabled implementation, one hosted Groq request, and four prompt-development runs on one demonstration job. Targets follow the success criteria in Section 1. They measure software behavior, not learning gains.

Table 9.1: Measured results against targets

Metric	Result	Target	What it shows
Core unit tests	20 / 20 passed	All tests pass	Validation, review gate, scoring, persistence and PDF paths work as designed.
Scripted UI checks	12 / 12 passed	All checks pass	Prepare, review, practice, progress and revision flows run end to end.
Hosted Groq request	6 drafts imported	At least 1 live request imports drafts	The deployed app can reach Groq and produce reviewable drafts.
Questions approved after human review	6 / 6	Every question reviewed before practice	The saved scripted UI report records all six genuine model questions approved through the review form. This measures the review workflow, not automatic validation accuracy.

Table 9.2: Prompt development results

Prompt version	Accepted	Inference time (Colab)	Recorded outcome
V1	1 / 6	27.48 s	Absent answer phrases, placeholder explanations and duplicate stem.
V2	2 / 6	31.96 s	Four answer phrases did not appear in evidence quotes.
V3	4 / 6	28.05 s	Incomplete option set and unsupported answer phrase.
V4	6 / 6	25.00 s	All six outputs passed implemented structural and evidence checks.

The local audit records 20 passing core tests and 12 passing scripted UI checks. The saved UI verification records six generated questions approved through the human review form; this is the evidence for the review-workflow metric below. The hosted Groq request is a point-in-time deployment check, not a continuous availability or accuracy guarantee. The V1 to V4 prompt results are reported from earlier Colab experiments. The public repository contains the Colab worker and a saved demo, but not the per-version raw outputs or timing log, so those figures cannot be independently reproduced from the repository alone. The listed inference times are from the earlier Colab setup, not the hosted Groq endpoint. All metrics describe software behavior, not learning gains.

10. Screenshots

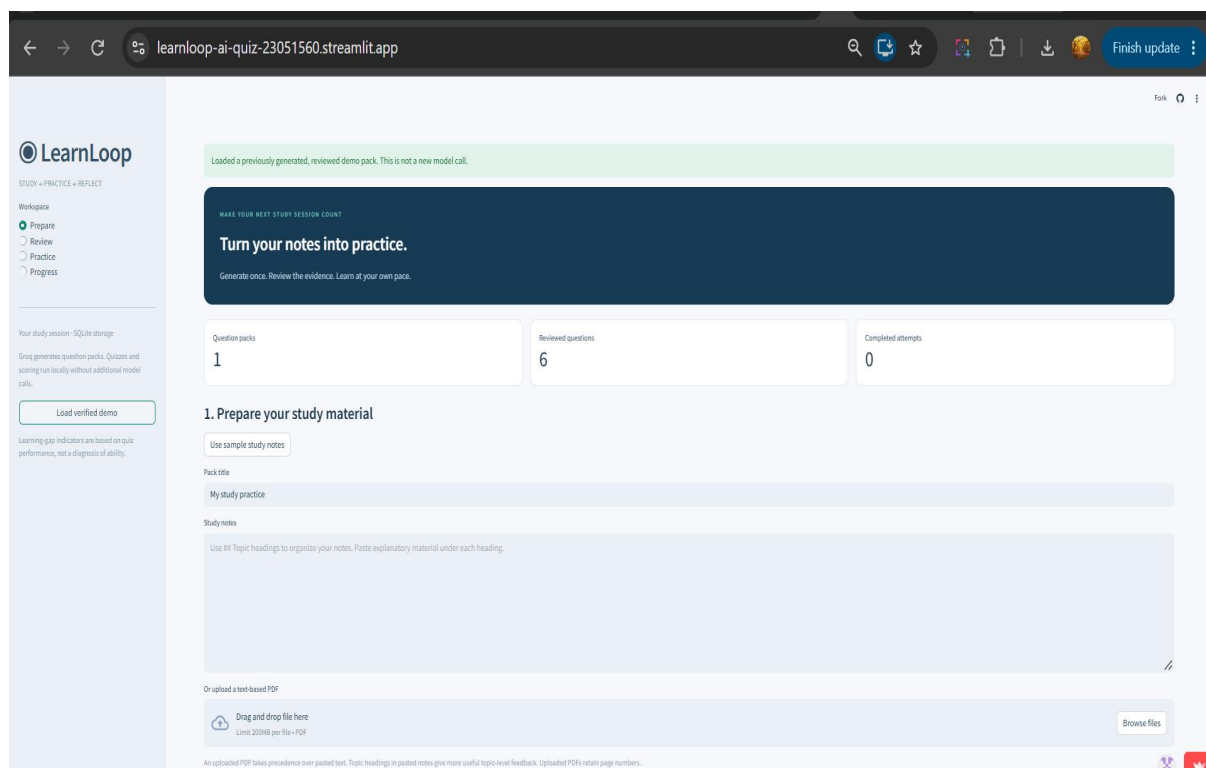


Figure 10.1: Prepare workspace with saved study pack and source input.

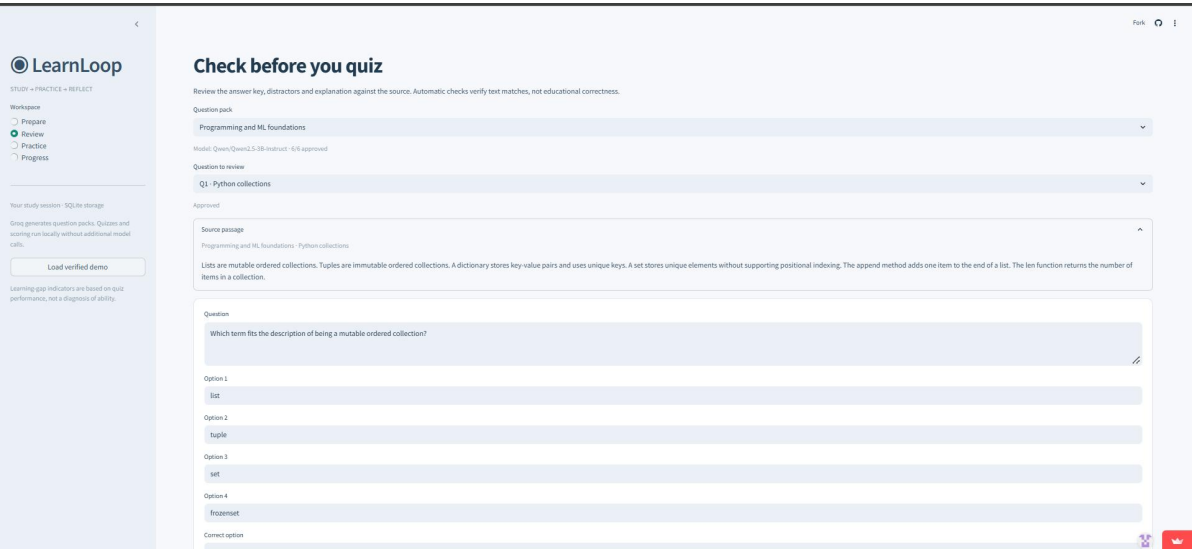


Figure 10.2: Review workspace showing source evidence and question approval.

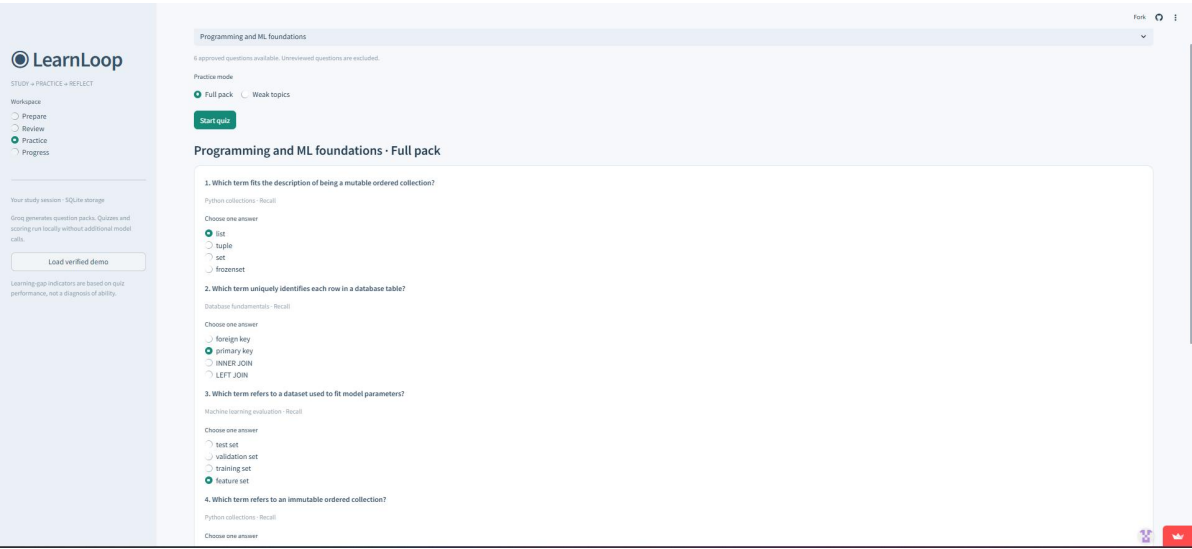


Figure 10.3: Practice workspace with the reviewed full question pack.

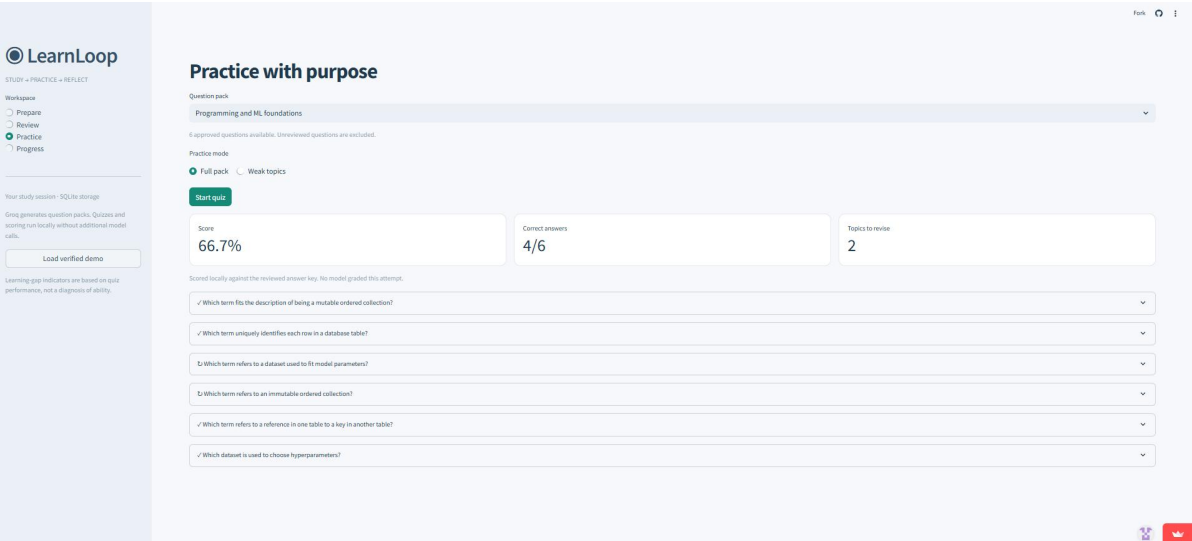


Figure 10.4: Quiz results showing score, correct answers, and topics to revise.

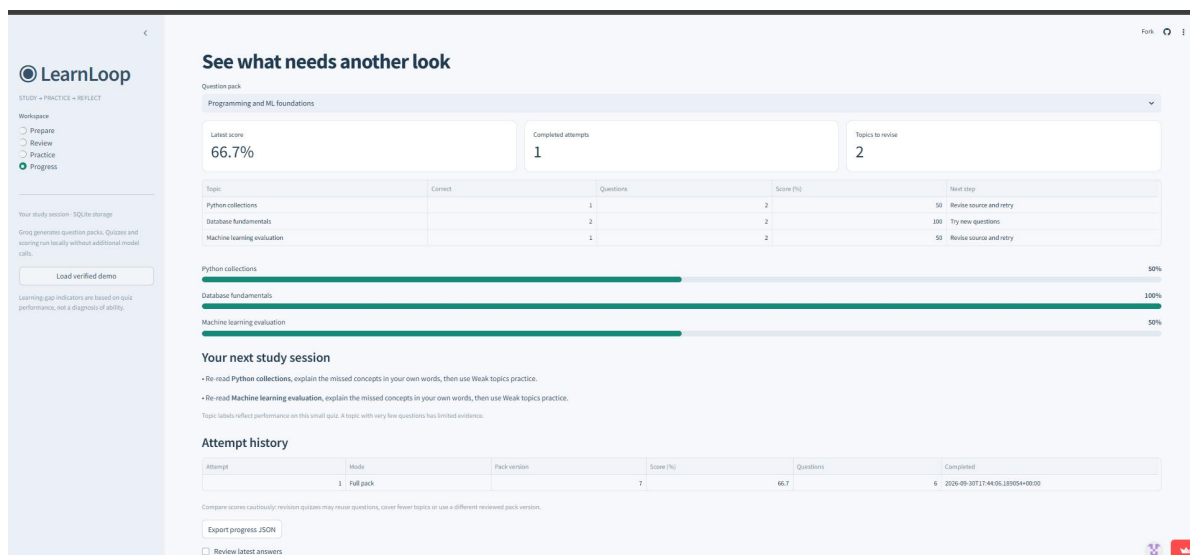


Figure 10.5: Progress workspace with topic performance and next study steps.

11. Challenges and Learnings

11.1 Challenges and Responses

Table 11.1: Challenges and responses

Challenge	Project response	Learning
Model output did not consistently follow JSON.	Added cleanup and strict field, option and evidence validation. Earlier Colab runs reported acceptance increasing from 1/6 to 6/6; the per-version outputs are not included in the public source repository.	Prompting helps format; deterministic validation adds a separate boundary.
Some outputs used unsupported answer phrases or placeholder explanations.	Check source quote and answer phrase, retain rejection reasons and require human approval.	Text overlap cannot judge semantic correctness.
Broad passage prompts produced repeated or vague questions.	Select a target fact and constrain answer form.	A narrower generation target is easier to validate.
Live hosting and credentials	The public repository excludes credentials; the Groq key is configured in Streamlit Secrets. A hosted Groq request generated six drafts. The Progress chart and low-contrast theme issues found during deployment checks were fixed and published.	Keep keys in server-side secrets; recheck service limits and hosted behavior before demonstrations.
Weak-topic practice reuses prior questions.	Warn that repeated-question scores do not prove mastery of unseen material.	Workflow success and learning effectiveness need separate evaluations.

11.2 Current Limitations and Unimplemented Capabilities

The table below distinguishes implemented behavior from capabilities the current release does not provide. These limits are part of the project scope and should be considered when evaluating the hosted demonstration.

Table 11.2: Current capability limits

Capability	Current status and limitation
OCR for scanned PDFs	Not implemented. PDF upload extracts embedded/selectable text with pypdf. Image-only pages are skipped; if no usable text remains, the app asks for OCR first. Apply OCR outside LearnLoop, then upload the searchable PDF.
PDF ingestion limits	Uploads are limited to 15 MB and 40 pages. Pages with fewer than 80 extracted characters are skipped, and a job accepts at most 30 source passages. PDF text extraction does not preserve page layout, tables or diagrams as visual structure.
Question correctness checks	Automated validation checks structure and text overlap with source evidence; it does not reason reliably about ambiguity, factual nuance or whether exactly one answer is educationally sound. A human must review every question before it enters practice.
Weak-topic practice	The current mode reuses reviewed questions from weaker topics. It does not create unseen questions, so repeat scores may reflect memory and are not evidence of transfer or mastery.
Accounts and cross-device history	There are no learner accounts, authentication, profile pages or cross-device synchronization. Browser visitors are isolated from each other, but their session data is not a durable personal learning record.
Hosted data retention	Hosted SQLite data is stored in a temporary per-session database. It is not backed up as permanent cloud storage and may be lost when the app session or hosting container is recycled. Export progress JSON if a copy is needed; exported jobs and packs can include study notes.
Exams and proctoring	Practice provides answer submission and scoring but is not a timed, secure or proctored examination system. It has no identity verification, invigilation or anti-cheating controls.
Groq availability and quotas	Question generation requires the configured server-side Groq key, network access, model availability and remaining account limits. There is no offline generation or alternate-provider fallback. Rate limits can reject requests; check the Groq Console for current limits and usage.
Availability after inactivity	Streamlit Community Cloud may hibernate the app after 12 hours without traffic. A visitor can wake it from the sleep screen; the first load may take longer. This is hosting behavior, not a fixed 30-day expiration.
Learning and diagnosis claims	Scores describe performance on the reviewed questions in a particular attempt. The 70% revision threshold is a transparent heuristic, not a validated measure of ability, diagnosis or learning gain. No controlled learner study has been completed.

12. Future Improvements

- **OCR for scanned notes:** add text recognition so image-only PDFs can be used without a separate OCR step.
- **Better PDF topic grouping:** group extracted passages by topic more precisely than the current heading-based split.
- **Durable, authenticated storage:** add learner accounts and persistent storage so history survives restarts and works across devices.
- **Unseen revision questions:** generate new questions for weak topics instead of reusing reviewed ones, so repeat scores reflect transfer.
- **Educator-led quality evaluation:** have teachers rate question quality, and calibrate difficulty and multi-step questions.

13. Conclusion

LearnLoop is a hosted Streamlit learning workflow that turns learner-provided notes into source-linked question drafts through Groq, requires human review before practice, scores attempts deterministically, and highlights topics for revision. The local audit passed 20 core tests and 12

scripted UI checks. A live hosted request generated six drafts with qwen/qwen3.8-27b through Groq. The hosted Progress chart and text-contrast issues were fixed and published. Text matching does not prove semantic correctness, and the demonstration does not establish learning gains.

These results support the tested software workflow and validation boundary. They do not establish general question accuracy, improved learning or uninterrupted Groq availability. The hosted app depends on Streamlit Community Cloud and Groq; availability, rate limits and model access can change. Scripted checks and the classroom demo are not a learning-outcome study.

14. References and Links

- Public GitHub repository: <https://github.com/AadiRatn/learnloop-ai-quiz>
- Hosted application: <https://learnloop-ai-quiz-23051560.streamlit.app/>
- Google Drive submission folder:
<https://drive.google.com/drive/u/0/folders/1NMAuyRourRThLHxq1QiwSIPTFrgHOTCV>
- Streamlit documentation: <https://docs.streamlit.io/>
- PyPDF documentation: <https://pypdf.readthedocs.io/>
- Groq API documentation: <https://console.groq.com/docs/>
- Streamlit Community Cloud hibernation: <https://docs.streamlit.io/deploy/streamlit-community-cloud/manage-your-app#app-hibernation>
- Google Colab: <https://colab.research.google.com/>